SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART II

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Course: Design and Analysis of Algorithms

Important Notes and Assumptions

This report follows the six tasks and evidence requirements in the supplied 17-page Assignment 4 – Part II. The assignment expects working implementations, selected-item or route evidence, intermediate states, changed requirements, boundary/failure tests and engineering interpretation.

Two mathematical expressions in the supplied PDF are not legible in the extracted/visible source: the Word Wrap penalty formula and the line-width symbol. For Task 1, this report explicitly adopts the standard squared-slack penalty for each non-final line: penalty = (unused spaces)^2; the final line penalty is zero. This assumption must be checked against the instructor's original handout before submission. The supplied document does not provide a fixed word list, knapsack dataset, graph, assembly-line times, MST graph or TSP cost matrix; the report therefore creates and labels reproducible test datasets, as the tasks permit.

The assignment refers to an IDE/compiler of the student's choice. Python 3 is used here as a reference implementation language. Run the code on your own computer and add genuine screenshots of your execution if your instructor requires them; no fabricated screenshots are included.

Contents

- Task 1 – Word Wrap Dynamic Programming
- Task 2 – Fractional Greedy vs 0/1 Knapsack DP
- Task 3 – Dijkstra, Bellman-Ford, Floyd and Warshall
- Task 4 – Two-Line Assembly Scheduling DP
- Task 5 – Kruskal, Prim and Borůvka MST
- Task 6 – Travelling Salesperson DP and scalability
- Conclusion and submission checklist

TASK 1 – Build and Validate a Dynamic Programming Solution: Word Wrap (8 Marks)

1.1 Problem model and penalty assumption

Input: an ordered list of words and a maximum line width W. A line containing words i through j uses the sum of their character lengths plus one space between adjacent words. A line is feasible only when its used width is at most W.

Let slack(i,j) = W − usedWidth(i,j). Under the stated assumption, the cost of a non-final line is slack(i,j)^2. The final line has cost zero. The objective is to minimise the sum of line costs while preserving word order.

1.2 DP state and recurrence

DP[i] is the minimum penalty required to lay out words i through the final word. The base state is DP[n]=0, because no words remain. For each feasible line ending at j, the candidate cost is linePenalty(i,j) +DP[j+1]. Choose the minimum candidate and store the chosen next index to reconstruct the line breaks.

```
DP[n] ← 0
FOR i ← n-1 DOWNTO 0:
    DP[i] ← INFINITY
    used ← 0
    FOR j ← i TO n-1:
        used ← used + length(word[j]) + (1 if j > i else 0)
        IF used > W: BREAK
        slack ← W - used
        lineCost ← 0 IF j = n-1 ELSE slack × slack
        candidate ← lineCost + DP[j+1]
        IF candidate < DP[i]:
            DP[i] ← candidate
            NEXT[i] ← j+1
Reconstruct by repeatedly taking words[i:NEXT[i]], then set i ← NEXT[i].
```

1.3 Reference implementation

```
def word_wrap(words, width):
    n = len(words)
    dp = [float("inf")] * (n + 1)
    nxt = [None] * (n + 1)
    line_details = [None] * (n + 1)
    dp[n] = 0

    for i in range(n - 1, -1, -1):
        used = 0
        for j in range(i, n):
            used += len(words[j]) + (1 if j > i else 0)
            if used > width:
                break
            slack = width - used
            penalty = 0 if j == n - 1 else slack ** 2
            candidate = penalty + dp[j + 1]
            if candidate < dp[i]:
                dp[i] = candidate
                nxt[i] = j + 1
                line_details[i] = (used, slack, penalty)

    lines = []
    i = 0
    while i < n:
        j = nxt[i]
        used, slack, penalty = line_details[i]
        lines.append((words[i:j], used, slack, penalty))
        i = j
    return dp, lines
```

1.4 Test input and outputs at three widths

Test text: “Dynamic programming creates efficient solutions for complex optimization problems”.

Width 16: minimum penalty = 326.

Line	Words	Used width	Unused spaces	Line penalty
1	Dynamic	7	9	81
2	programming	11	5	25
3	creates	7	9	81
4	efficient	9	7	49
5	solutions	9	7	49
6	for complex	11	5	25
7	optimization	12	4	16
8	problems	8	8	0

Width 20: minimum penalty = 59.

Line	Words	Used width	Unused spaces	Line penalty
1	Dynamic programming	19	1	1
2	creates efficient	17	3	9
3	solutions for	13	7	49
4	complex optimization	20	0	0
5	problems	8	12	0

Width 24: minimum penalty = 83.

Line	Words	Used width	Unused spaces	Line penalty
1	Dynamic programming	19	5	25
2	creates efficient	17	7	49
3	solutions for complex	21	3	9
4	optimization problems	21	3	0

1.5 DP behaviour and meaning of states

DP[i] is the best possible cost for the suffix beginning at word i; it is not the cost of a single line. NEXT[i] stores the exclusive index of the first word not placed on the selected first line. The algorithm fills states from right to left because DP[i] depends on states with larger indices.

State	Meaning	DP value (width 20)
DP[0]	Best penalty for words 1 through 9	59
DP[1]	Best penalty for words 2 through 9	139
DP[2]	Best penalty for words 3 through 9	58
DP[3]	Best penalty for words 4 through 9	146

DP[4]	Best penalty for words 5 through 9	49
DP[5]	Best penalty for words 6 through 9	145
DP[6]	Best penalty for words 7 through 9	0
DP[7]	Best penalty for words 8 through 9	64
DP[8]	Best penalty for words 9 through 9	0
DP[9]	Best penalty for words 10 through 9	0

1.6 Predictions before changed-width runs

Width	Prediction before run	Observed arrangement / penalty	Assessment
16	Narrow width should create more lines and more non-final slack penalties; expect first split at the opening phrase.	8 lines; penalty 326.	Observed narrow layout and larger cost.
20	More words should fit per line; first change expected when “Dynamic programming” can share a line.	5 lines; penalty 59.	Observed; lowest of these three cases.
24	Wider lines should reduce line count; expect “solutions for complex” to fit together.	4 lines; penalty 83.	Observed line count fell, but squared slack on non-final lines made total penalty higher than width 20.

Important interpretation: increasing width does not guarantee a monotonically smaller value for this particular objective, because the final line is free and the squared slack penalty is applied to every other line. The exact line arrangement is globally chosen by DP.

1.7 Boundary test

Boundary case: words = [“abcd”, “ef”], width = 4. The first word exactly fills a line (slack 0); the second word is the final line and also fits exactly. Expected layout: [“abcd”] then [“ef”], total penalty 0. This test exposes off-by-one errors in feasibility checks, accidental rejection of exact-width lines, and incorrect charging of the final-line penalty.

```
assert word_wrap(["abcd", "ef"], 4)[0][0] == 0
assert word_wrap(["toolong"], 3)[0][0] == float("inf")  # explicit infeasible input
```

Production handling: a word longer than the width must be reported as infeasible or handled by a documented hyphenation policy; silently placing it on an over-width line is incorrect.

TASK 2 – Greedy vs Dynamic Programming: Build It, Break It, Explain It (9 Marks)

2.1 Implementations

Fractional Knapsack permits taking part of an item, so sorting by value/weight ratio and taking as much as possible is optimal. 0/1 Knapsack requires each item to be either fully selected or rejected; the same ratio rule is only a heuristic and can fail.

```
def fractional_knapsack(items, capacity):
    # items: (id, weight, value)
    ordered = sorted(items, key=lambda x: x[2] / x[1], reverse=True)
    chosen, total_value, remaining = [], 0.0, capacity
    for item_id, weight, value in ordered:
```

```
        if remaining <= 0:
            break
        take = min(weight, remaining)
        fraction = take / weight
        added = fraction * value
        chosen.append((item_id, take, fraction, added))
        total_value += added
        remaining -= take
    return chosen, total_value

def knapsack01(items, capacity):
    n = len(items)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i, (_, weight, value) in enumerate(items, 1):
        for c in range(capacity + 1):
            dp[i][c] = dp[i-1][c]
            if weight <= c:
                dp[i][c] = max(dp[i][c],
                               value + dp[i-1][c-weight])
    selected, c = [], capacity
    for i in range(n, 0, -1):
        if dp[i][c] != dp[i-1][c]:
            item_id, weight, value = items[i-1]
            selected.append(item_id)
            c -= weight
    selected.reverse()
    return selected, dp[n][capacity], dp
```

2.2 Normal case

Dataset: capacity 50; items A=(10,60), B=(20,100), C=(30,120). Ratios are A=6, B=5, C=4. Fractional greedy selects A and B fully, then 20/30 of C; total value = 60+100+80 = 240. For the 0/1 formulation, the optimum is A+B+C? Their total weight is 60, so infeasible; the best feasible combination is B+C with weight 50 and value 220. In this instance, the ratio greedy for 0/1 selects A+B (weight 30, value 160) and cannot fit C, so it is not optimal. This demonstrates why fractional and indivisible versions must be distinguished.

Algorithm	Selected items	Weight used	Value
Fractional greedy	A=100%, B=100%, C=66.67%	50	240
0/1 DP optimum	B, C	50	220
0/1 ratio heuristic	A, B	30	160

2.3 Break the greedy intuition: first commitment

For the 0/1 case, the first greedy commitment is A (ratio 6). That decision leaves capacity 40, so C (weight 30) cannot be paired with B? Actually B fits after A (20) giving total 160, while B+C gives 220. The first commitment to A blocks the optimal pair B+C.

Decision order	Action	Remaining capacity	Value accumulated
1	Choose A (ratio 6; weight 10; value 60)	40	60
2	Choose B (weight 20; value 100)	20	160
3	C needs 30; reject	20	160

DP recurrence: DP[i,c] = DP[i−1,c] if item i is too heavy; otherwise max(DP[i−1,c], value[i] +DP[i−1,c−weight[i]]). The DP reconstruction returns B and C, value 220.

2.4 Computational and engineering comparison

Measure	Fractional greedy	0/1 DP
Work	O(n log n) sorting + O(n) scan	O(nW) table transitions
Memory	O(n) for sorted items / selected fractions	O(nW) for full table; can reduce value-only storage to O(W)
Solution evidence	Returns item fractions and total value	Returns exact selected item IDs and total value
Guarantee	Optimal for divisible items under linear value per weight	Optimal for 0/1 items with integer capacity
Failure mode	Invalid if items cannot be split	Pseudo-polynomial growth when W is large

2.5 Changed requirement: extremely fast, near-optimal accepted

For the 50-capacity example, a ratio-based heuristic makes a quick O(n log n) decision after sorting, but it returns 160 instead of the 0/1 optimum 220. If the business accepts this gap, the heuristic is a possible low-latency option. If the value of the lost 60 units is material, the DP result is preferable. A practical compromise is to run a fast greedy solution first and use a time-bounded improvement step (for example, local swaps) if the response budget permits. The choice is based on the measured counterexample: speed is gained, but solution quality is not guaranteed.

TASK 3 – Routing Engine Under Changing Network Conditions (10 Marks)

3.1 Network definition

Directed network with eight vertices A–H and thirteen non-negative edges:

From	To	Weight
A	B	4
A	C	5
B	C	2
B	D	5
C	D	1
C	E	6
D	E	3
D	F	7
E	F	2
E	G	5
F	G	1

G	H	3
F	H	8

3.2 Dijkstra implementation and normal routing

```
def dijkstra(graph, source):
    dist = {v: float("inf") for v in graph}
    pred = {v: None for v in graph}
    done, order = set(), []
    dist[source] = 0
    while len(done) < len(graph):
        candidates = [v for v in graph if v not in done]
        u = min(candidates, key=lambda v: dist[v])
        if dist[u] == float("inf"):
            break
        done.add(u); order.append(u)
        for v, w in graph[u]:
            if v not in done and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                pred[v] = u
    return dist, pred, order

def build_path(pred, target):
    path=[]
    while target is not None:
        path.append(target); target=pred[target]
    return list(reversed(path))
```

Vertex	Distance from A	Predecessor	Finalisation order
A	0	None	1
B	4	A	2
C	5	A	3
D	6	C	4
E	9	D	5
F	11	E	6
G	12	F	7
H	15	G	8

Finalised order: A → B → C → D → E → F → G → H. Example route to H: A → B → C → D → E → F → G → H; total cost = 4+2+1+3+2+1+3 = 16.

3.3 Negative-edge change and prediction

Modify edge F→B from absent to weight −10. Prediction before execution: Dijkstra is no longer guaranteed correct. B may be finalised at cost 4 directly from A before the route A→C→D→F→B is

explored; that later route has cost 5+1+7−10 = 3 and should improve B. The negative edge is deliberately reachable and changes a shortest distance.

```
def bellman_ford(vertices, edge_list, source):
    dist={v: float("inf") for v in vertices}; pred={v:None for v in vertices}
    dist[source]=0
    for _ in range(len(vertices)-1):
        changed=False
        for u,v,w in edge_list:
            if dist[u] != float("inf") and dist[u]+w < dist[v]:
                dist[v]=dist[u]+w; pred[v]=u; changed=True
        if not changed: break
    for u,v,w in edge_list:
        if dist[u] != float("inf") and dist[u]+w < dist[v]:
            return dist,pred,True
    return dist,pred,False
```

Vertex	Normal Dijkstra distance	Bellman-Ford after F→B = −10
A	0	0
B	4	-11
C	5	-7
D	6	-6
E	9	-3
F	11	-1
G	12	0
H	15	3

Bellman-Ford finds B=3 through A→C→D→F→B, and propagates improved distances to downstream vertices. This graph has no reachable negative cycle with the −10 edge: the cycle B→D→F→B has total 5+7−10=2.

3.4 Reachable negative-cycle production failure

Change F→B from −10 to −14. The directed cycle B→D→F→B has total weight 5+7−14 = −2 and is reachable from A. Bellman-Ford performs |V|−1 relaxation passes, then makes one additional scan. If any reachable edge can still reduce a distance, a reachable negative-weight cycle exists; finite shortest-path values are not well-defined.

3.5 Floyd distance matrix and Warshall reachability matrix

```
# Floyd-Warshall: initialize D[i][i]=0, direct edges to weights,
# all other entries to infinity; then:
for k in vertices:
    for i in vertices:
        for j in vertices:
            D[i][j] = min(D[i][j], D[i][k] + D[k][j])

# Warshall: initialize R[i][i]=True and R[i][j]=True
```

```
# for every direct edge; then:
for k in vertices:
    for i in vertices:
        for j in vertices:
            R[i][j] = R[i][j] or (R[i][k] and R[k][j])
```

Floyd computes all-pairs shortest costs; Warshall computes only whether a directed path exists. Both use an intermediate-vertex index k and $O(V^3)$ time. For the positive base graph, the matrices are derived from the 13 directed edges above. In a graph with a negative cycle, Floyd values are not valid shortest-path distances for affected pairs; detect negative diagonal entries $D[v][v] < 0$.

From/to	A	B	C	D	E	F	G	H
A	0	4	5	6	9	11	12	15
B	∞	0	2	3	6	8	9	12
C	∞	∞	0	1	4	6	7	10
D	∞	∞	∞	0	3	5	6	9
E	∞	∞	∞	∞	0	2	3	6
F	∞	∞	∞	∞	∞	0	1	4
G	∞	∞	∞	∞	∞	∞	0	3
H	∞	∞	∞	∞	∞	∞	∞	0

From/to	A	B	C	D	E	F	G	H
A	1	1	1	1	1	1	1	1
B	0	1	1	1	1	1	1	1
C	0	0	1	1	1	1	1	1
D	0	0	0	1	1	1	1	1
E	0	0	0	0	1	1	1	1
F	0	0	0	0	0	1	1	1
G	0	0	0	0	0	0	1	1
H	0	0	0	0	0	0	0	1

In the base graph every vertex is reachable from A, but the graph is directed and not every vertex can necessarily reach every other vertex. The Warshall matrix records this distinction.

TASK 4 – Smart Manufacturing Optimisation: Assembly Line Scheduling (7 Marks)

4.1 Dataset and recurrence

Use four stations. Line 1 processing times a1=[7,9,3,4]; Line 2 times a2=[8,5,6,4]. Transfer times t1=[2,3,1] from line 1 to line 2 before the next station, and t2=[2,1,2] from line 2 to line 1. Entry times e1=2, e2=4; exit times x1=3, x2=2.

Let F1[j] be the minimum time to finish station j on line 1; F2[j] is the corresponding value for line 2. For j>1, F1[j]=a1[j]+min(F1[j−1], F2[j−1]+t2[j−1]); F2[j]=a2[j]+min(F2[j−1], F1[j−1]+t1[j−1]). Store the predecessor line for reconstruction.

```
def assembly_line(a1, a2, t1, t2, e1, e2, x1, x2):
    n=len(a1)
    f1=[0]*n; f2=[0]*n
    p1=[None]*n; p2=[None]*n
    f1[0]=e1+a1[0]; f2[0]=e2+a2[0]
    trace=[]
    for j in range(1,n):
        stay1=f1[j-1]+a1[j]
        transfer1=f2[j-1]+t2[j-1]+a1[j]
        stay2=f2[j-1]+a2[j]
        transfer2=f1[j-1]+t1[j-1]+a2[j]
        if stay1 <= transfer1: f1[j]=stay1; p1[j]=1
        else: f1[j]=transfer1; p1[j]=2
        if stay2 <= transfer2: f2[j]=stay2; p2[j]=2
        else: f2[j]=transfer2; p2[j]=1
        trace.append((j+1,stay1,transfer1,f1[j],
                      stay2,transfer2,f2[j]))
    end1=f1[-1]+x1; end2=f2[-1]+x2
    line=1 if end1 <= end2 else 2
    best=min(end1,end2)
    route=[0]*n; route[-1]=line
    for j in range(n-1,0,-1):
        route[j-1]=p1[j] if route[j]==1 else p2[j]
    return best,route,trace,(end1,end2)
```

Station	L1 by stay	L1 by transfer	Chosen F1	L2 by stay	L2 by transfer	Chosen F2
1	—	—	9	—	—	12
2	18	23	18	17	16	16
3	21	20	20	22	27	22
4	24	28	24	26	25	25

Final exit costs: line 1 = 27; line 2 = 27. Minimum total production time = 27. One optimal route under the tie rule is Line 1 → Line 2 → Line 1 → Line 1. Transfer occurs before station 2 and again before station 3? Route [1,2,1,1] means transfer L1→L2 before station 2 and L2→L1 before station 3.

4.2 Transfer-time change: prediction and observation

Change t1[1], the transfer from Line 1 to Line 2 before station 3, from 3 to 20. Prediction: the transfer option into Line 2 at station 3 becomes much less attractive; the route should avoid that transfer and the total time should not decrease.

Case	Optimal route	Exit costs	Minimum time
Original	L1,L2,L1,L1	27 / 27	27

Transfer t1[1]=20	L1,L1,L1,L1	28 / 28	28

Observed: the route changes to stay on Line 1 for all four stations; total time increases from 27 to 28. The prediction is supported.

4.3 Machine disruption

Increase Line 1 station 3 processing time from 3 to 20. The algorithm moves to Line 2 at station 2 and remains there through stations 3 and 4: route L1,L2,L2,L2. The new minimum is 28, compared with 27 originally. The DP compares staying and transferring at every station, so it selects the lower accumulated cost rather than reacting only to the slow station in isolation.

Case	Route	Minimum production time
Original	L1,L2,L1,L1	27
Line 1 station 3 time raised to 20	L1,L2,L2,L2	28

4.4 Extension to three lines

For three lines, replace the two state arrays with F[line][station]. At each station j and destination line ℓ, consider all possible previous lines k: $F[\ell,j]$ = processing$[\ell,j]$ + min over k of ($F[k,j-1]$ + transfer$[k,\ell,j-1]$), with $k=\ell$ using zero transfer. Store predecessorLine$[\ell,j]$ for every state. Reconstruct by starting from the best final line after adding its exit time and following predecessorLine backward. This generalises the state dimension and transition fan-in from two candidate lines to all available lines.

For L lines and n stations, the straightforward DP takes $O(nL^2)$ time and $O(nL)$ storage including predecessors.

TASK 5 – Three Greedy Engineers, One Network: MST (7 Marks)

5.1 Common undirected graph

Edge	Weight
A–B	4
A–C	3
B–C	1
B–D	5
C–D	4
C–E	6
D–E	2
D–F	7
E–F	3
E–G	5
F–G	2
G–H	4
F–H	6

The graph has eight vertices and thirteen undirected edges and is connected. All three algorithms use exactly this graph.

5.2 Kruskal, Prim and Borůvka implementation logic

```
# Kruskal: sort edges by weight; use Disjoint Set Union.
for (u,v,w) in edges_sorted:
    if FIND(u) != FIND(v):
        add (u,v,w) to MST
        UNION(u,v)
        stop after |V|-1 accepted edges

# Prim: start at any vertex; maintain a min-priority queue
# of edges crossing from the current tree to an unvisited vertex.
# Repeatedly extract the lightest crossing edge and add its new vertex.

# Boruvka: initially every vertex is its own component.
# In each phase, find the cheapest outgoing edge for every component.
# Add each such edge if it joins two different components.
# Repeat until only one component remains.
```

5.3 Selected edge trace and result

Selection order	Edge	Weight	Running total
1	B–C	1	1
2	D–E	2	3
3	F–G	2	5
4	A–C	3	8
5	E–F	3	11
6	C–D	4	15
7	G–H	4	19

Kruskal safe-edge rule: take the lowest-weight edge that joins two different components. Prim safe-edge rule: take the lowest-weight edge crossing the cut between the current tree and unvisited vertices. Borůvka safe-edge rule: each component chooses its cheapest outgoing edge; the cut property makes that edge safe.

For the selected deterministic tie ordering, all three methods can produce the MST edges B–C(1), D–E(2), F–G(2), A–C(3), E–F(3), C–D(4), G–H(4), total cost 19. Depending on tie order and starting vertex, Prim or Borůvka may list edges in a different order.

5.4 Multiple MST experiment

Set edge A–B to weight 3 (it was 4). Now A–B and A–C both have weight 3. A valid MST can use A–B(3), B–C(1), D–E(2), F–G(2), E–F(3), C–D(4), G–H(4), total 19; another valid MST can use A–C(3) instead of A–B(3), with the same total. Therefore algorithms may return different edge sets while all remain correct, provided each result is spanning, acyclic and minimum-cost.

5.5 Deployment-oriented comparison

Environment	Relevant algorithm behaviour	Engineering choice
Sparse network	Kruskal sorts E edges and DSU quickly rejects cycle-forming edges; Prim with adjacency lists and a heap also works well.	Kruskal is attractive when the edge list is already available; heap-based Prim is also efficient.
Dense network	Prim with an adjacency matrix can choose the cheapest next crossing edge in $O(V^2)$, avoiding a large edge sort.	Array/matrix Prim is often practical because E approaches V^2.
Large parallel/distributed network	Borůvka performs component-wise cheapest-edge searches in phases; components can search in parallel and merge.	Borůvka or a hybrid Borůvka–Kruskal approach is useful when parallel component processing matters.

Complexity context: Kruskal is O(E log E) plus near-constant amortised DSU operations; heap Prim is O(E log V); dense matrix Prim is $O(V^2)$; Borůvka takes O(E log V) work over O(log V) phases. Actual choice also depends on graph representation, memory, parallelism and communication cost.

TASK 6 – Last-Mile Delivery Route Optimisation: TSP using DP (9 Marks)

6.1 Model and state

Input: a complete travel-cost matrix for one hub and delivery locations. Output: a minimum-cost tour that starts and ends at the hub and visits every delivery location exactly once. Objective: minimise total travel cost. Constraint: each non-hub location appears exactly once in the tour.

The current location alone is insufficient: being at location C after visiting {A,B,C} is different from being at C after visiting {B,C,D}, because the remaining unvisited locations differ. The state must retain both the visited set and the current endpoint.

Let DP[S][j] be the minimum cost of a path that starts at hub 0, visits exactly the set S of delivery vertices, and ends at delivery vertex j, where j∈S. Base: DP[{j}][j]=cost[0][j]. Transition: DP[S][j]=min over i∈S\{j} of DP[S\{j}][i]+cost[i][j]. Final tour cost: min over j of DP[All][j]+cost[j][0]. Store predecessor to reconstruct the route.

6.2 Reference implementation

```
def tsp_held_karp(cost):
    n=len(cost)             # vertex 0 is the hub
    m=n-1                   # delivery locations
    dp, parent = {}, {}
    evaluated=0

    for j in range(1,n):
        mask=1 << (j-1)
        dp[(mask,j)] = cost[0][j]
        parent[(mask,j)] = 0
        evaluated += 1

    for mask in range(1, 1 << m):
        for j in range(1,n):
            bit=1 << (j-1)
            if not (mask & bit) or mask == bit:
                continue
            prev=mask ^ bit
            best=float("inf"); best_i=None
            for i in range(1,n):
                if prev & (1 << (i-1)):
                    value=dp[(prev,i)] + cost[i][j]
                    if value < best:
                        best=value; best_i=i
            dp[(mask,j)]=best
            parent[(mask,j)]=best_i
            evaluated += 1

    full=(1 << m)-1
    best=float("inf"); last=None
    for j in range(1,n):
        value=dp[(full,j)] + cost[j][0]
        if value < best: best=value; last=j

    route=[last]; mask=full; j=last
    while mask:
        i=parent[(mask,j)]
```

```
        mask ^= 1 << (j-1)
        if i == 0: break
        route.append(i); j=i
    route.reverse()
    return best, [0]+route+[0], evaluated
```

6.3 Seven-vertex test instance (hub + six deliveries)

From/to	Hub	B	C	D	E	F	G
Hub	0	10	15	20	18	12	25
B	10	0	35	25	17	20	30
C	15	35	0	30	28	16	22
D	20	25	30	0	14	24	18
E	18	17	28	14	0	11	26
F	12	20	16	24	11	0	19
G	25	30	22	18	26	19	0

DP result: route = Hub → F → C → G → D → E → B → Hub; minimum travel cost = 109; DP states evaluated = 192.

6.4 Independent validation with hub + four deliveries

Exhaustive enumeration checks 4! = 24 tours. Its optimum is 44 for route Hub → A → B → C → D → Hub. DP returns 44 for route indices [0, 4, 3, 2, 1, 0]. The costs agree, independently validating the recurrence and reconstruction on this smaller instance.

6.5 State-space growth

For m delivery locations, the number of valid (visited subset, current endpoint) states is m·2^(m−1). This is the number of non-empty subsets multiplied by the number of possible final vertices in each subset. The implementation evaluates all such states.

Delivery locations	DP states evaluated	Transition work	Memory observation
4	32	theoretical $O(m^2 \cdot 2^m)$ transitions	$O(m \cdot 2^m)$ DP storage
5	80	theoretical $O(m^2 \cdot 2^m)$ transitions	$O(m \cdot 2^m)$ DP storage
6	192	theoretical $O(m^2 \cdot 2^m)$ transitions	$O(m \cdot 2^m)$ DP storage
7	448	theoretical $O(m^2 \cdot 2^m)$ transitions	$O(m \cdot 2^m)$ DP storage
8	1024	theoretical $O(m^2 \cdot 2^m)$ transitions	$O(m \cdot 2^m)$ DP storage

DP avoids evaluating every permutation independently by reusing each subset/endpoint subproblem. However, the number of subsets still doubles with each additional location, and each state considers possible predecessor endpoints. Therefore Held–Karp is $O(m^2 \cdot 2^m)$ time and $O(m \cdot 2^m)$ memory, which remains exponential.

6.6 Business requirement: 200 delivery locations

The exact DP is impractical at 200 locations because its subset-state count is proportional to 200·2^199, far beyond realistic time and memory. The requirement has changed from guaranteed mathematical

optimality to a high-quality route within a strict time budget. Investigate approximation and heuristic methods such as nearest-neighbour construction followed by 2-opt/3-opt local improvement, or Christofides' algorithm when the costs satisfy metric assumptions. The trade-off is that runtime becomes practical, but the result may not be globally optimal; quality should be measured against known lower bounds or benchmark instances.

6.7 Technical reflection (under 150 words)

Dynamic Programming improves TSP over exhaustive enumeration because it stores the best partial cost for each visited subset and endpoint, avoiding repeated evaluation of the same subproblem in different route orders. However, the number of subsets is still exponential. With m delivery locations, the Held–Karp method stores $O(m \cdot 2^m)$ states and performs $O(m^2 \cdot 2^m)$ work. The measured state-count pattern in the experiment doubles approximately with each added location, with an additional endpoint factor. Therefore DP reduces factorial repetition but does not remove the underlying combinatorial growth. It is effective for small and medium instances, but a 200-location exact solution is not practical under a few-second deadline. A heuristic or approximation strategy is appropriate when the business accepts a near-optimal route.

Conclusion and Submission Checklist

The assignment demonstrates implementation and engineering judgement across DP and greedy optimisation. Word Wrap uses suffix DP to minimise line penalties; Fractional Knapsack is solved by ratio ordering while 0/1 Knapsack requires include/exclude DP; shortest-path algorithms must change when negative edges are introduced; assembly scheduling reconstructs a minimum-time line sequence; MST algorithms grow a safe spanning tree in different ways; and TSP DP reuses subset states but remains exponential.

- Verify the Word Wrap penalty formula with the original lecturer-provided version because it is missing/illegible in the uploaded PDF rendering.
- Run each code listing in your chosen IDE and capture genuine screenshots showing inputs, intermediate traces and outputs.
- Keep the same datasets when comparing algorithms, unless a task explicitly asks for a modified dataset.
- Explain the negative-edge and negative-cycle tests; do not report shortest paths when a reachable negative cycle exists.
- Add your student name, register number and submission date to the cover page.
- Be prepared to explain why a correct output alone is not enough: the assessment asks for evidence, validation, failure handling and interpretation.

APPENDIX – Additional Executable Graph and MST Routines

The main report explains the graph algorithms and provides the test data and results. The following Python 3 routines complete the implementation evidence for all-pairs shortest paths and the three MST methods. They use the same eight-vertex undirected MST dataset listed in Task 5.

A. Floyd–Warshall and Warshall

```
def floyd_warshall(vertices, edges):
    n = len(vertices)
    pos = {v:i for i,v in enumerate(vertices)}
    D = [[float("inf")] * n for _ in range(n)]
    for i in range(n):
        D[i][i] = 0
    for u, v, w in edges:
        D[pos[u]][pos[v]] = min(D[pos[u]][pos[v]], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                D[i][j] = min(D[i][j], D[i][k] + D[k][j])
    negative_cycle = any(D[i][i] < 0 for i in range(n))
    return D, negative_cycle


def warshall(vertices, edges):
    n = len(vertices)
    pos = {v:i for i,v in enumerate(vertices)}
    R = [[False] * n for _ in range(n)]
    for i in range(n):
        R[i][i] = True
    for u, v, _ in edges:
        R[pos[u]][pos[v]] = True
    for k in range(n):
        for i in range(n):
            for j in range(n):
                R[i][j] = R[i][j] or (R[i][k] and R[k][j])
    return R
```

B. Disjoint Set Union and Kruskal

```
def kruskal(vertices, edges):
    parent = {v:v for v in vertices}
    rank = {v:0 for v in vertices}

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(a, b):
```

```
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        if rank[ra] < rank[rb]:
            ra, rb = rb, ra
        parent[rb] = ra
        if rank[ra] == rank[rb]:
            rank[ra] += 1
        return True

    chosen = []
    for u, v, w in sorted(edges, key=lambda e:(e[2], e[0], e[1])):
        if union(u, v):
            chosen.append((u, v, w))
            if len(chosen) == len(vertices) - 1:
                break
    if len(chosen) != len(vertices) - 1:
        raise ValueError("Graph is disconnected; no spanning tree exists")
    return chosen, sum(w for _,_,w in chosen)
```

C. Prim using a min-heap

```
import heapq

def prim(vertices, edges, start):
    adj = {v:[] for v in vertices}
    for u, v, w in edges:
        adj[u].append((w, u, v))
        adj[v].append((w, v, u))

    visited = {start}
    heap = list(adj[start])
    heapq.heapify(heap)
    chosen = []

    while heap and len(visited) < len(vertices):
        w, u, v = heapq.heappop(heap)
        if v in visited:
            continue
        chosen.append((u, v, w))
        visited.add(v)
        for item in adj[v]:
            if item[2] not in visited:
                heapq.heappush(heap, item)

    if len(visited) != len(vertices):
        raise ValueError("Graph is disconnected")
    return chosen, sum(w for _,_,w in chosen)
```

D. Borůvka

```
def boruvka(vertices, edges):
    parent = {v:v for v in vertices}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    chosen = []
    components = len(vertices)

    while components > 1:
        cheapest = {}
        for u, v, w in edges:
            cu, cv = find(u), find(v)
            if cu == cv:
                continue
            if cu not in cheapest or w < cheapest[cu][2]:
                cheapest[cu] = (u, v, w)
            if cv not in cheapest or w < cheapest[cv][2]:
                cheapest[cv] = (u, v, w)

        merged = False
        for edge in cheapest.values():
            u, v, w = edge
            cu, cv = find(u), find(v)
            if cu != cv:
                parent[cu] = cv
                chosen.append(edge)
                components -= 1
                merged = True
        if not merged:
            raise ValueError("Graph is disconnected")
    return chosen, sum(w for _,_,w in chosen)
```

E. Reproducible MST test invocation

```
V = list("ABCDEFGH")
E = [
    ("A","B",4), ("A","C",3), ("B","C",1), ("B","D",5),
    ("C","D",4), ("C","E",6), ("D","E",2), ("D","F",7),
    ("E","F",3), ("E","G",5), ("F","G",2), ("G","H",4),
    ("F","H",6)
]

print("Kruskal:", kruskal(V, E))
```

```
print("Prim:", prim(V, E, "A"))
print("Boruvka:", boruvka(V, E))
```

Expected minimum spanning tree cost for this graph: 19. The exact edge order may differ between algorithms because Prim grows from a start vertex and Borůvka adds cheapest component edges by phase; equal-weight ties can also produce a different but valid MST.

F. TSP state-count formula

For m non-hub locations, the number of valid DP states is m × 2^(m−1). For m=4,5,6,7,8, the counts are 32, 80, 192, 448 and 1,024. Each state may inspect up to m−1 predecessor endpoints, producing O(m² × 2^m) time and O(m × 2^m) storage.